Non-Primitive Data Types

Mastering Reference Types and Objects in Java

Non-primitive types are known as **Reference Types**. Unlike primitives that store their data "in-place," these variables store a memory address that **points (refers)** to an object in the computer's memory.

1. The Four Major Differences

Custom vs Built-in

Primitives are part of Java. Non-primitives are created by the coder (except String).

Methods & Actions

Non-primitives can call methods!

```
name.toUpperCase()
```

Naming Style

Primitives: lowercase (int, char)
Non-Prim: Uppercase (String, Array)

The "Null" Ability

Non-primitives can be **null**, meaning they point to nothing yet.

2. Common Reference Types

Strings

The most common non-primitive. Used to store text.

```
String name = "Alice";
System.out.println(name.length()); // Method call!
```

Arrays

A single variable that stores multiple values of the same type.

```
int[] numbers = {1, 2, 3, 4, 5};
```

Classes & Objects

Custom blue-prints created by programmers to represent real objects like Cars or Users.

```
Car myCar = new Car();
```

3. Why use Non-Primitives?

- **Flexibility:** They can handle complex data structures.
- **Functionality:** Built-in methods let you manipulate data easily.
- **Memory:** They use dynamic memory, making them scalable.
- **OOP:** They are the foundation of Object-Oriented Programming (Classes).

🚀 Pro Summary

- Reference types = Non-Primitive types.
- They store **addresses**, not values.
- They start with an **Uppercase** letter.

codehive

- They can call **methods**.
- Examples: **String, Array, Class, Enum**.

codehive